



Stacks

Deep Dive + Real-World Use Cases + Complexity

Java 8+



Session Structure

1. Stack fundamentals
2. Implementation approaches
3. Time & space complexity
4. Real-world use cases
5. Monotonic stack pattern
6. Interview-level problems



What is a Stack?

A stack follows:

LIFO (Last In, First Out)

Example:

Push: 1, 2, 3

Pop → 3



Real World Example

- Browser back button
- Undo/Redo systems
- Call stack in recursion
- Expression evaluation
- Syntax parsing



Basic Operations

Operation	Description	Time
push()	Insert element	$O(1)$
pop()	Remove top	$O(1)$
peek()	View top	$O(1)$
isEmpty()	Check empty	$O(1)$

Stack Using Array

```
class ArrayStack {
    int[] arr;
    int top = -1;
    int capacity;

    ArrayStack(int size) {
        capacity = size;
        arr = new int[size];
    }

    void push(int val) {
        if (top == capacity - 1)
            throw new RuntimeException("Overflow");
        arr[++top] = val;
    }

    int pop() {
        if (top == -1)
            throw new RuntimeException("Underflow");
        return arr[top--];
    }

    int peek() {
        if (top == -1)
            throw new RuntimeException("Empty");
        return arr[top];
    }
}
```



Stack Using Deque (Recommended)

Java 8+ best practice:

```
java.util.Deque<Integer> stack = new java.util.ArrayDeque<>();  
  
stack.push(10);  
stack.push(20);  
int top = stack.pop();
```

Time: $O(1)$



Why Not Use `java.util.Stack`?

- Legacy class
- Extends Vector
- Synchronized (slower)

Prefer: **`ArrayDeque`**



Space Complexity

If n elements:

Space = $O(n)$

Recursive stack calls also use stack memory.



Real World Use Case — Browser Back Button

When user navigates:

Push current page to stack.

When user presses back:

Pop and load previous.

Time per operation: $O(1)$



Real World Use Case — Undo/Redo

Each action:

Push onto undo stack.

Undo:

Pop from undo → push to redo stack.



Balanced Parentheses Problem

Given:

"{[()]}"

Check if valid.

Time: $O(n)$

Space: $O(n)$

Valid Parentheses Code

```
static boolean isValid(String s) {  
    java.util.Deque<Character> stack = new java.util.ArrayDeque<>();  
  
    for (char c : s.toCharArray()) {  
        if (c == '(' || c == '{' || c == '[')  
            stack.push(c);  
        else {  
            if (stack.isEmpty()) return false;  
            char top = stack.pop();  
            if ((c == ')' && top != '(') ||  
                (c == '}' && top != '{') ||  
                (c == ']' && top != '['))  
                return false;  
        }  
    }  
    return stack.isEmpty();  
}
```



Expression Evaluation (Postfix)

Example:

"2 3 + 4 *"

Time: $O(n)$

Space: $O(n)$

Postfix Evaluation Code

```
static int evaluatePostfix(String[] tokens) {
    java.util.Deque<Integer> stack = new java.util.ArrayDeque<>();

    for (String token : tokens) {
        if ("+-*/".contains(token)) {
            int b = stack.pop();
            int a = stack.pop();
            switch (token) {
                case "+": stack.push(a + b); break;
                case "-": stack.push(a - b); break;
                case "*": stack.push(a * b); break;
                case "/": stack.push(a / b); break;
            }
        } else {
            stack.push(Integer.parseInt(token));
        }
    }
    return stack.pop();
}
```



Monotonic Stack Pattern

Used for:

- Next Greater Element
- Largest Rectangle in Histogram
- Stock Span

Next Greater Element

Time: $O(n)$

Space: $O(n)$

```
static int[] nextGreater(int[] nums) {
    int n = nums.length;
    int[] result = new int[n];
    java.util.Deque<Integer> stack = new java.util.ArrayDeque<>();

    for (int i = n - 1; i >= 0; i--) {
        while (!stack.isEmpty() && stack.peek() <= nums[i])
            stack.pop();

        result[i] = stack.isEmpty() ? -1 : stack.peek();
        stack.push(nums[i]);
    }
    return result;
}
```



Stack vs Recursion

Recursion uses implicit stack.

Example:

```
static void recursive(int n) {  
    if (n == 0) return;  
    recursive(n - 1);  
}
```

Each call uses stack memory.



Common Mistakes

- Not checking empty before pop
- Overflow in fixed-size implementation
- Using Stack instead of Deque
- Forgetting stack becomes empty at end



Interview Discussion Points

- Why is stack $O(1)$?
- When to use monotonic stack?
- Stack vs Queue difference?
- Recursion vs explicit stack?



Practice Problems

Easy:

- Reverse string using stack
- Valid parentheses

Medium:

- Min stack
- Evaluate infix expression

Advanced:

- Largest rectangle in histogram
- Trapping rain water (stack version)



Summary

Stacks provide:

- LIFO behavior
- $O(1)$ push/pop
- Core to parsing & recursion
- Powerful monotonic patterns

Master:

- Valid parentheses
- Postfix evaluation
- Next greater element
- Monotonic stack logic